Decorators

Wrapping functions with extra powers

Shwetank's Python Journey • Timer • Debugger • Cache

What is a Decorator?

A decorator is a function that takes another function as input, adds some extra behaviour around it, and returns a new function — without modifying the original function's code. It is Python's way of following the Open/Closed Principle: open for extension, closed for modification.

The @ syntax is just shorthand. These two are exactly equivalent:

```
# Using @ syntax (clean way)
@my_decorator
def greet():
    print('Hello!')

# Without @ (what Python actually does under the hood)
def greet():
    print('Hello!')
greet = my_decorator(greet)    # ← same thing!
```

NOTE A decorator relies on two concepts from the Scope & Closures notes: (1) functions are first-class objects — they can be passed as arguments. (2) Closures — the inner wrapper function 'remembers' the original func.

Decorator Anatomy

Step	Code Pattern
1 Define decorator	`def my_decorator(func):`
2 Define wrapper inside	`    def wrapper(*args, **kwargs):`
3 Add logic BEFORE	`        # run before original`
4 Call original	`        result = func(*args, **kwargs)`
5 Add logic AFTER	`        # run after original`
6 Return result	`        return result`
7 Return wrapper	`    return wrapper`
8 Apply with @	`@my_decorator`

> **TIP** Always use *args and **kwargs in your wrapper function. This makes the decorator work on ANY function regardless of how many parameters it has.

① Timing Function Execution — *Timer Decorator*

Measure exactly how long a function takes to run, without touching the function's own code. The decorator records time before and after the function executes and prints the difference.

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()            # record start time
        result = func(*args, **kwargs) # run the original function
        end = time.time()              # record end time
        print(f'{func.__name__} ran in {end - start:.4f} seconds')
        return result                  # don't forget to return!
    return wrapper


@timer
def make_chai(cups):
    time.sleep(0.5)   # simulating work
    return f'{cups} cups of chai ready'


result = make_chai(3)
print(result)
```

▶ **Output**

```
make_chai ran in 0.5003 seconds
3 cups of chai ready
```

> **NOTE** func.__name__ gives you the original function's name (make_chai), not 'wrapper'. This is very useful for logging. We'll see how to preserve this properly with @functools.wraps below.

> **TIP** Real use case: use the timer decorator on database queries or API calls to spot slow operations without adding timing code all over your codebase.

? Interview Q: Why do we return result from the wrapper instead of just calling func()?

☑ If we only called func() without returning its value, the decorator would swallow the return value — the caller would always get None. Always capture and return the result so the decorated function behaves identically to the original.

② Debugging Function Calls *— Debug Decorator*

Automatically log every call to a function — printing the function name and all argument values. This is far better than sprinkling print() statements everywhere by hand.

```
def debug(func):
    def wrapper(*args, **kwargs):
        # Build a readable args string
        args_repr   = [repr(a) for a in args]
        kwargs_repr = [f'{k}={repr(v)}' for k, v in kwargs.items()]
        all_args    = ', '.join(args_repr + kwargs_repr)

        print(f'Calling {func.__name__}({all_args})')
        result = func(*args, **kwargs)
        print(f'{func.__name__} returned {repr(result)}')
        return result
    return wrapper


@debug
def add(a, b):
    return a + b

@debug
def greet(name, greeting='Hello'):
    return f'{greeting}, {name}!'


add(10, 20)
greet('Shwetank', greeting='Namaste')
```

▶ **Output**

```
Calling add(10, 20)
add returned 30
Calling greet('Shwetank', greeting='Namaste')
greet returned 'Namaste, Shwetank!'
```

NOTE	repr() is used instead of str() because repr() shows the type clearly — e.g. 'Shwetank' (with quotes) instead of Shwetank. This makes it obvious that a value is a string, not an integer.

TIP	Stack multiple decorators! @debug on top of @timer means every call gets both timed and logged — decorators compose naturally.

? Interview Q: How does **kwargs work in the debug decorator's output?

☑ kwargs.items() gives (key, value) pairs. We format each as 'k=repr(v)' so keyword arguments show up exactly as they were typed: greeting='Namaste'. Then we join positional and keyword args together with a comma.

③ Cache Return Values — *Memoisation Decorator*

Store the result of expensive function calls and return the cached result when the same arguments are used again. This technique is called memoisation — trading memory for speed.

```
def cache(func):
    cache_store = {}   # lives in closure — shared across all calls

    def wrapper(*args):
        if args in cache_store:
            print(f'[CACHE HIT]  {func.__name__}{args}')
            return cache_store[args]   # return stored result

        print(f'[COMPUTING]  {func.__name__}{args}')
        result = func(*args)           # run the real function
        cache_store[args] = result     # store result for later
        return result
    return wrapper


@cache
def slow_square(n):
    import time
    time.sleep(1)   # simulate heavy computation
    return n * n


print(slow_square(5))  # computed (slow)
print(slow_square(5))  # from cache (instant)
print(slow_square(7))  # computed (slow)
print(slow_square(7))  # from cache (instant)
```

▶ **Output**

```
[COMPUTING]  slow_square(5,)
25
[CACHE HIT]  slow_square(5,)
25
[COMPUTING]  slow_square(7,)
49
[CACHE HIT]  slow_square(7,)
49
```

NOTE cache_store = {} is defined in the cache function's scope — this is a closure variable. Each decorated function gets its own separate cache_store dictionary, so @cache on two different functions doesn't mix their results.

TIP Python's standard library already has this! Use @functools.lru_cache(maxsize=128) for a production-ready cache with a size limit. Our version above is how it works internally.

WARN Our cache uses args (a tuple) as the dictionary key. This only works if all arguments are hashable (strings, numbers, tuples). Passing a list as an argument will raise a TypeError because lists are not hashable.

? Interview Q: What problem does memoisation solve, and when would you use it?

☑ It solves redundant recomputation. Classic use cases: Fibonacci sequence, recursive algorithms, expensive API calls, or any pure function (same input always gives same output). Don't use it for functions with side effects or functions whose output changes over time.

🔑 Bonus: @functools.wraps — The Missing Piece

Without @functools.wraps, the decorated function loses its identity — its __name__ and __doc__ become those of wrapper, not the original function. Always add it.

```
import functools

def timer(func):
    @functools.wraps(func)   # ← preserves func.__name__, __doc__, etc.
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f'{func.__name__} ran in {time.time() - start:.4f}s')
        return result
    return wrapper
```

```
@timer
def make_chai(): pass

print(make_chai.__name__)   # 'make_chai' ☑  (not 'wrapper')
```

TIP	Make @functools.wraps a habit. Add it to every wrapper function you write. It takes one second and saves confusion when debugging or using help() on decorated functions.

Summary — Three Decorator Patterns

Decorator	Core Mechanism	Real-world Use
`@timer`	time.time() before & after	Profile slow DB queries, API calls, algorithms
`@debug`	repr(args) + repr(kwargs) logging	Trace bugs without print-spamming your code
`@cache`	dict keyed on args tuple (closure)	Speed up recursive functions, expensive computations

Decorators are closures in a tuxedo — same idea, just dressed up with @.